# Resultados experimentales: Bubble Sort y Quicksort

Héctor Manuel Salazar Rivera
Optimización y Métodos Computacionales

Noviembre de 2026

## 1. Alcance

Este documento reúne la metodología, los casos de prueba, las gráficas y la interpretación del punto 3 de los entregables. El análisis formal está en `section1_comparison/activity1_formal_analysis/`: el PDF del examen y un anexo que añade la terminación de los dos algoritmos, la corrección de Quicksort y el análisis del pivote por mediana de 3. Los datos completos están en `section3_deliverables/data/` y las gráficas en `section3_deliverables/charts/`. Las tablas y las cifras de este texto se generan desde esos archivos con `section3_deliverables/build_results_report.py`.

## 2. Implementación evaluada

Ambos algoritmos están en `section1_comparison/activity2_implementation/algorithms.py` y no usan funciones de ordenamiento de Python. Una prueba revisa que el código no las llame.

**Bubble Sort** es iterativo y termina antes si una pasada no hace intercambios. Esa salida temprana produce el mejor caso de $n-1$ comparaciones que da el examen. El pseudocódigo del examen no la incluye: sin ella, un arreglo ordenado también haría $n(n-1)/2$ comparaciones.

**Quicksort** es recursivo y parte el arreglo con dos punteros, $i$ y $j$, que avanzan el uno hacia el otro y se detienen en elementos iguales al pivote. El pivote es la mediana de los elementos en las posiciones $lo$, $\lfloor (lo+hi)/2 \rfloor$ y $hi$.

## 3. Elección del pivote: mediana de 3

El examen analiza como peor caso un arreglo inverso con el último elemento como pivote: cada partición deja un lado vacío y $T(n) = T(n-1) + cn$, es decir $\Theta(n^2)$. Con $n = 4000$, eso significa unas $n^2/2 = 8\,000\,000$ comparaciones. Esta cifra sale de la fórmula del examen y no se midió, porque el programa no implementa ese pivote.

Con mediana de 3, en un arreglo ordenado o inverso el elemento central es la mediana exacta, y ese caso se vuelve el mejor. Con $n = 4000$ e inverso se midieron 48 115 comparaciones, unas 166 veces menos, y una profundidad de 12, igual a $\log_2 n$ redondeado. El costo es de 2 o 3 comparaciones adicionales por llamada.

La mediana de 3 no elimina el caso cuadrático, solo hace que sea difícil encontrarlo por accidente. Para comprobarlo, el experimento incluye una entrada construida con el adversario de McIlroy (1999), que decide el orden de los valores durante las comparaciones para perjudicar siempre al pivote. Con esa entrada Quicksort crece como $n^2$ y su profundidad llega a 2 001 con $n = 4000$,

cerca de $n/2$. Se prefirió la mediana de 3 frente a un pivote aleatorio porque es determinista: la misma entrada da siempre la misma ejecución, lo que permite repetir los experimentos y reproducir esa ejecución en la animación.

## 4. Metodología

**Tamaños.** $n = 250, 500, 1000, 2000, 4000, 8000, 16000$. Al duplicar el tamaño en cada paso, la razón $T(2n)/T(n)$ se lee directamente: cerca de 4 para $n^2$ y cerca de 2 para $n$ o $n \log n$. Los puntos también quedan a igual distancia en un eje logarítmico. Por debajo de 250 los tiempos más cortos se acercan al ruido de medición: Bubble Sort sobre un arreglo ordenado con $n = 250$ ya tarda menos de 5 microsegundos. El tope es 16000 porque ahí Bubble Sort ya tarda entre 4 y 7 s por corrida y el experimento completo toma unos 7 minutos. Duplicar el tamaño una vez más cuadruplicaría el tiempo de Bubble Sort.

**Repeticiones.** 7 por combinación, para un total de 588 corridas cronometradas. Se reporta la mediana porque una interrupción del sistema operativo solo puede alargar una corrida y la mediana no se deja arrastrar por ese valor. El número es impar para que la mediana sea una medición real. La dispersión justifica este número: desde $n = 2000$ el coeficiente de variación más alto es 0.062. El más alto de todo el experimento es 0.29, en una corrida de unos 24 microsegundos ($n = 1000$).

**Configuraciones.** Aleatorio con reales, aleatorio con enteros, ordenado, inverso, muy repetidos (10 valores distintos) y adversario de mediana de 3. El enunciado pide aleatorio, ordenado e inverso. Los repetidos y el adversario se agregaron para ver los límites del pivote.

**Control de la medición.** En `section1_comparison/activity3_experiment/experiment.py`, la función `measure` cronometra solo la llamada al algoritmo con `time.perf_counter`. Los datos se generan antes y se copian fuera del cronómetro. La interfaz no participa: los datos guardados vienen de `run_experiment.py`, que corre sin ventana. Los dos algoritmos reciben el mismo arreglo en cada repetición, y su orden se alterna para que ninguno aproveche siempre la memoria caché que deja el otro. El recolector de basura se ejecuta antes de cada corrida y queda apagado durante ella. La semilla es 42.

**Conteo de operaciones.** Las comparaciones, los intercambios y la profundidad se cuentan en corridas aparte, sin cronómetro, porque contar agrega costo. Se usan los tamaños 250, 500, 1000, 2000, 4000 con 3 repeticiones, y se reporta la mediana.

**Equipo.** CPython 3.14.8 en Linux 7.2.10-arch1-1, x86_64. El experimento completo tardó 424 s. El archivo `environment.txt` no registró el modelo del procesador.

## 5. Casos de prueba

Las pruebas están en `section1_comparison/activity2_implementation/test_algorithms.py` y `section2_animation/test_replay.py`. Son 26 y pasan todas con el comando que indica el README. Cubren:

- Arreglos fijos con enteros, reales, mezcla de ambos, repetidos, negativos y todos iguales, además de ordenado, inverso, vacío, un elemento, dos elementos y tres casos de 5000 elementos. Cada resultado se revisa: debe quedar ordenado y conservar los mismos elementos.
- 500 listas aleatorias con enteros y reales mezclados, por algoritmo.
- La mediana de 3 en cada orden posible de tres valores, y la división por la mitad en arreglos ordenados e inversos.

- Las fórmulas del examen: $n - 1$ comparaciones en Bubble Sort ordenado y $n(n-1)/2$ en inverso.
- La fidelidad de la animación: los eventos que dibuja reproducen el resultado real, y sus contadores coinciden con un conteo independiente.

# 6. Resultados

**Tabla 1:** Bubble Sort: mediana del tiempo en milisegundos, por tamaño $n$.

| Configuración | 250 | 500 | 1000 | 2000 | 4000 | 8000 | 16000 |
|---|---|---|---|---|---|---|---|
| Aleatorio (reales) | 0.766 | 3.78 | 19.3 | 84.7 | 354 | 1430 | 5845 |
| Aleatorio (enteros) | 0.761 | 3.77 | 18.9 | 83.2 | 347 | 1412 | 5759 |
| Ordenado | 0.00477 | 0.0113 | 0.0236 | 0.0493 | 0.0995 | 0.198 | 0.403 |
| Inverso | 0.896 | 4.42 | 22.6 | 103 | 439 | 1840 | 7413 |
| Muy repetidos (10 valores) | 0.727 | 3.61 | 18.1 | 82.8 | 340 | 1413 | 5684 |
| Adversario de mediana de 3 | 0.474 | 2.49 | 12.9 | 56.8 | 238 | 970 | 3926 |

**Tabla 2:** Quicksort: mediana del tiempo en milisegundos, por tamaño $n$.

| Configuración | 250 | 500 | 1000 | 2000 | 4000 | 8000 | 16000 |
|---|---|---|---|---|---|---|---|
| Aleatorio (reales) | 0.0949 | 0.236 | 0.511 | 1.05 | 2.23 | 4.84 | 10.4 |
| Aleatorio (enteros) | 0.0982 | 0.233 | 0.518 | 1.06 | 2.26 | 4.81 | 10.3 |
| Ordenado | 0.0439 | 0.104 | 0.23 | 0.503 | 1.09 | 2.35 | 5.01 |
| Inverso | 0.049 | 0.119 | 0.26 | 0.535 | 1.14 | 2.44 | 5.17 |
| Muy repetidos (10 valores) | 0.0821 | 0.198 | 0.426 | 0.9 | 1.9 | 4.11 | 8.89 |
| Adversario de mediana de 3 | 0.218 | 1.06 | 4.21 | 16.9 | 67.7 | 270 | 1106 |

**Tabla 3:** Cuántas veces tarda más Bubble Sort que Quicksort, por tamaño $n$. Un valor menor que 1 indica que Bubble Sort es más rápido.

| Configuración | 250 | 500 | 1000 | 2000 | 4000 | 8000 | 16000 |
|---|---|---|---|---|---|---|---|
| Aleatorio (reales) | 8.07 | 16.1 | 37.8 | 80.7 | 159 | 295 | 562 |
| Aleatorio (enteros) | 7.75 | 16.2 | 36.5 | 78.3 | 154 | 293 | 558 |
| Ordenado | 0.109 | 0.109 | 0.102 | 0.0982 | 0.0911 | 0.0844 | 0.0805 |
| Inverso | 18.3 | 37.2 | 86.9 | 192 | 385 | 754 | 1435 |
| Muy repetidos (10 valores) | 8.86 | 18.2 | 42.5 | 92 | 179 | 343 | 639 |
| Adversario de mediana de 3 | 2.17 | 2.34 | 3.07 | 3.35 | 3.51 | 3.6 | 3.55 |

**Tabla 4:** Ajuste $T(n) = c\,n^k$ por mínimos cuadrados en escala log-log. La columna $k$ todo usa todos los tamaños; $k$ grande y $k$ teórica usan $n \geq 2000$. La razón $T(2n)/T(n)$ compara $n = 16000$ con $n = 8000$. $R^2$ corresponde al ajuste con todos los tamaños.

| Algoritmo | Configuración | Teoría | $k$ todo | $k$ grande | $k$ teórica | $R^2$ | $T(2n)/T(n)$ | teórica |
|---|---|---|---|---|---|---|---|---|
| Bubble Sort | Aleatorio (reales) | $n^2$ | 2.14 | 2.03 | 2.00 | 0.999 | 4.09 | 4.00 |
| Bubble Sort | Aleatorio (enteros) | $n^2$ | 2.14 | 2.04 | 2.00 | 0.999 | 4.08 | 4.00 |
| Bubble Sort | Ordenado | $n$ | 1.05 | 1.01 | 1.00 | 0.999 | 2.04 | 2.00 |
| Bubble Sort | Inverso | $n^2$ | 2.17 | 2.06 | 2.00 | 0.999 | 4.03 | 4.00 |
| Bubble Sort | Muy repetidos (10 valores) | $n^2$ | 2.15 | 2.04 | 2.00 | 0.999 | 4.02 | 4.00 |
| Bubble Sort | Adversario de mediana de 3 | $n^2$ | 2.16 | 2.04 | 2.00 | 0.999 | 4.04 | 4.00 |
| Quicksort | Aleatorio (reales) | $n \log n$ | 1.11 | 1.10 | 1.12 | 0.999 | 2.15 | 2.15 |
| Quicksort | Aleatorio (enteros) | $n \log n$ | 1.11 | 1.09 | 1.12 | 0.999 | 2.15 | 2.15 |
| Quicksort | Ordenado | $n \log n$ | 1.13 | 1.11 | 1.12 | 1.000 | 2.14 | 2.15 |
| Quicksort | Inverso | $n \log n$ | 1.11 | 1.09 | 1.12 | 0.999 | 2.12 | 2.15 |
| Quicksort | Muy repetidos (10 valores) | $n \log n$ | 1.11 | 1.10 | 1.12 | 1.000 | 2.16 | 2.15 |
| Quicksort | Adversario de mediana de 3 | $n^2$ | 2.03 | 2.01 | 2.00 | 1.000 | 4.10 | 4.00 |

**Tabla 5:** Operaciones contadas con $n = 4000$ (mediana de 3 repeticiones).

| | Bubble Sort | | Quicksort | | |
|---|---|---|---|---|---|
| Configuración | Comparaciones | Intercambios | Comparaciones | Intercambios | Profundidad |
| Aleatorio (reales) | 7 995 984 | 4 033 288 | 67 058 | 11 568 | 24 |
| Aleatorio (enteros) | 7 995 299 | 3 979 005 | 68 491 | 11 439 | 24 |
| Ordenado | 3 999 | 0 | 48 105 | 0 | 12 |
| Inverso | 7 998 000 | 7 998 000 | 48 115 | 2 000 | 12 |
| Muy repetidos (10 valores) | 7 922 145 | 3 575 218 | 58 192 | 18 161 | 18 |
| Adversario de mediana de 3 | 5 997 000 | 2 662 731 | 4 019 986 | 1 999 | 2 001 |

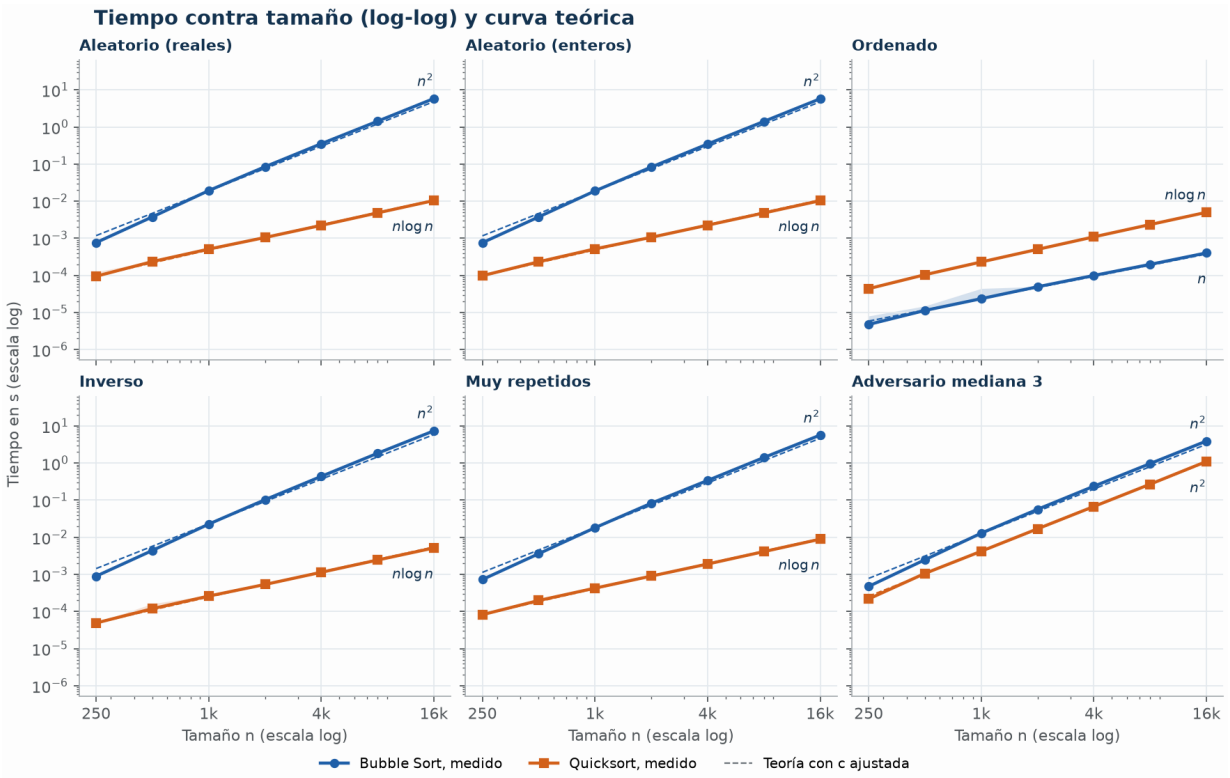



**Figura 1:** Tiempo contra tamaño en escala log-log. La línea punteada es la curva teórica con su constante ajustada a las mediciones. La banda va del mínimo al máximo. Solo en el panel del adversario las dos curvas tienen la misma complejidad teórica, $n^2$.

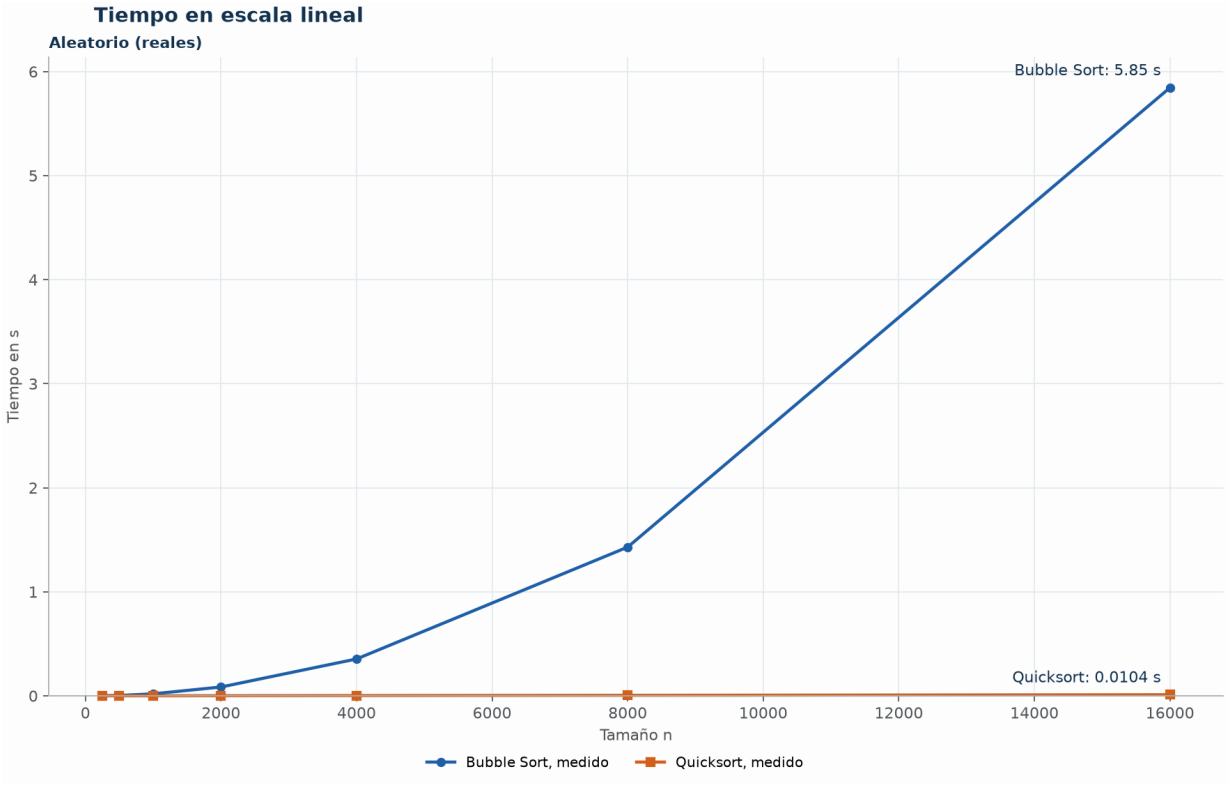



**Figura 2:** Arreglo aleatorio en escala lineal. Con $n = 16000$, Bubble Sort tarda 5.85 s y Quicksort 10.4 ms, tan poco que su curva casi no se separa de cero.

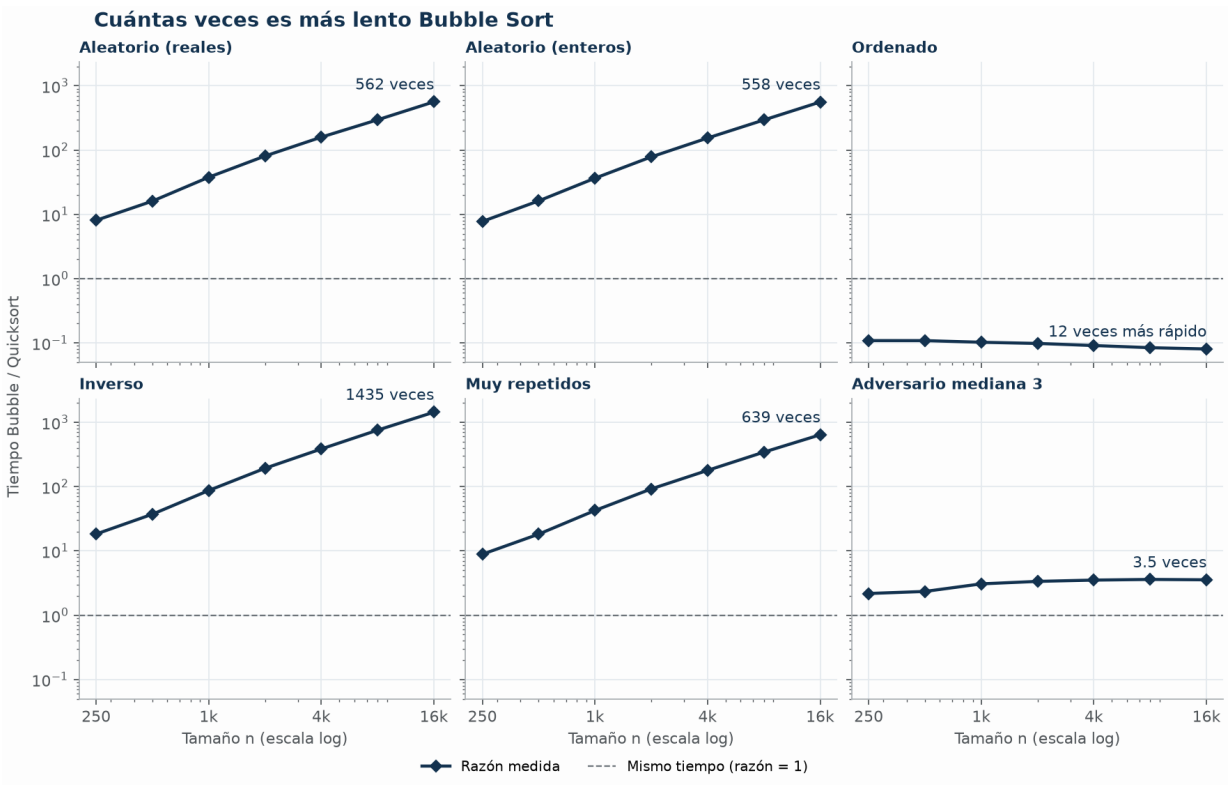



**Figura 3:** Razón de tiempos. Crece con $n$ en cuatro paneles. En ordenado está por debajo de 1 y en el adversario se estabiliza cerca de 3.5.

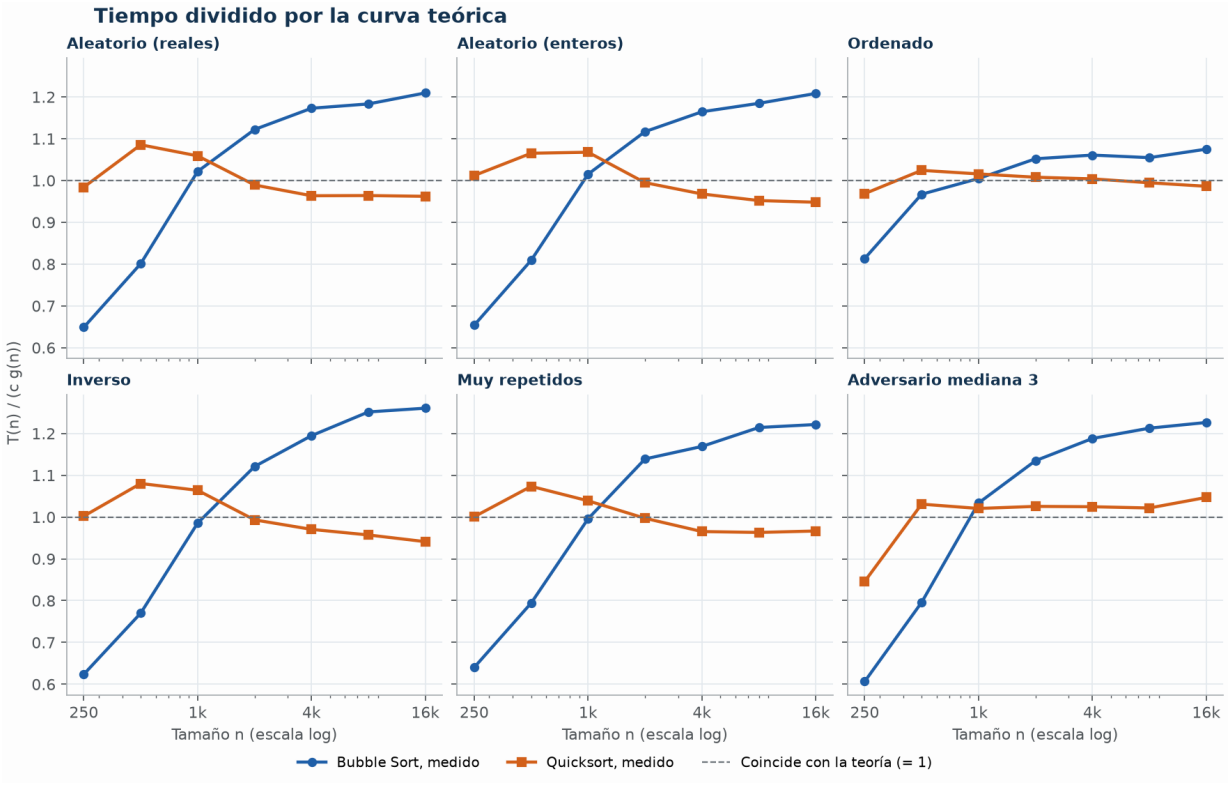



**Figura 4:** Tiempo dividido entre la curva teórica ajustada. Una curva plana indica el crecimiento esperado. Bubble Sort sube en los tamaños pequeños y se aplana desde $n = 2000$ aproximadamente.

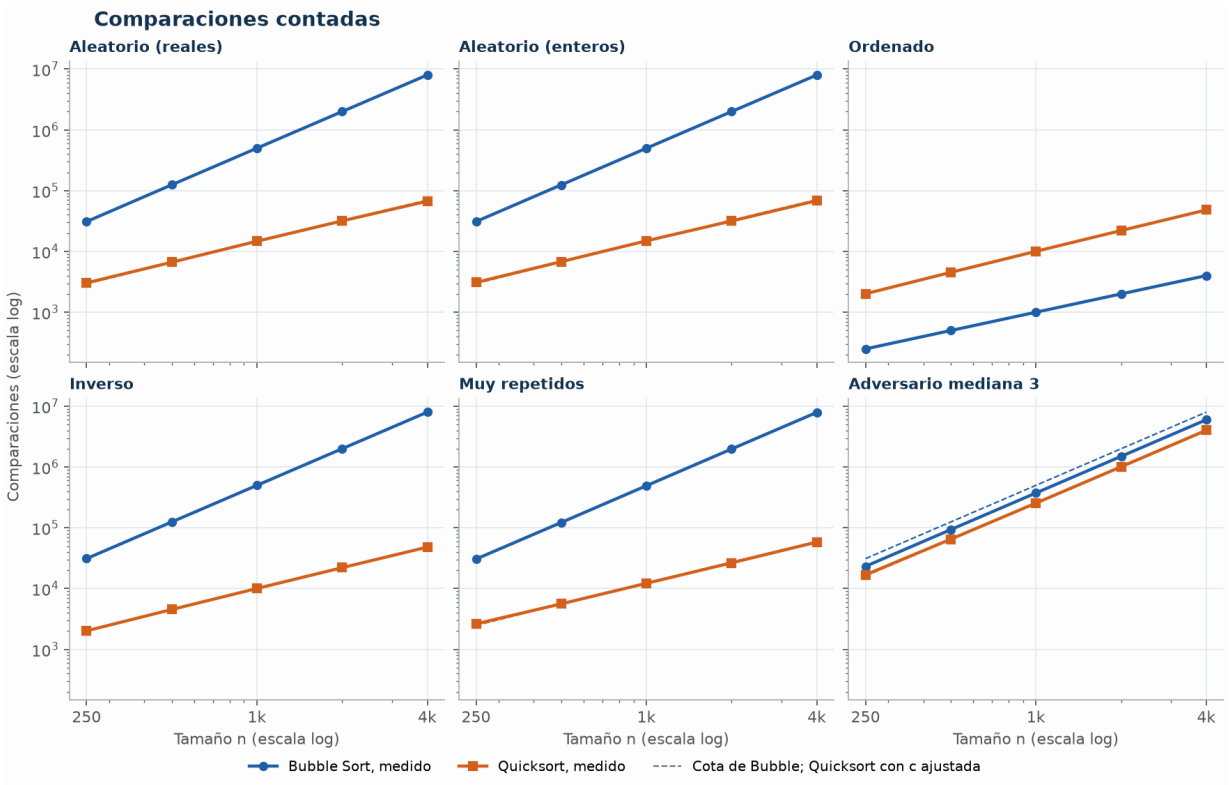



**Figura 5:** Comparaciones contadas. Las líneas punteadas de Bubble Sort son las fórmulas del examen: $n-1$ en ordenado y, como máximo, $n(n-1)/2$ en los demás. Las de Quicksort tienen la constante ajustada.

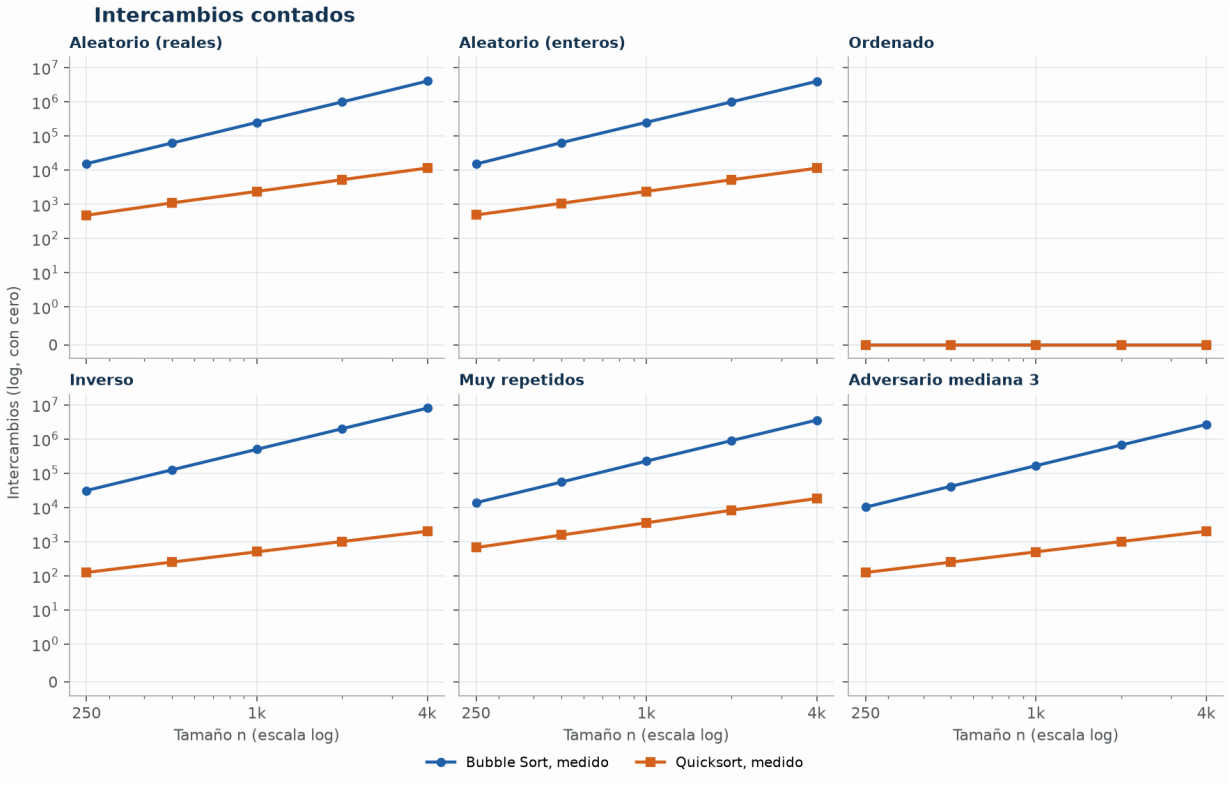



**Figura 6:** Intercambios contados. En un arreglo ordenado ninguno de los dos intercambia. El eje es logarítmico salvo cerca de cero, para poder mostrar ese valor.

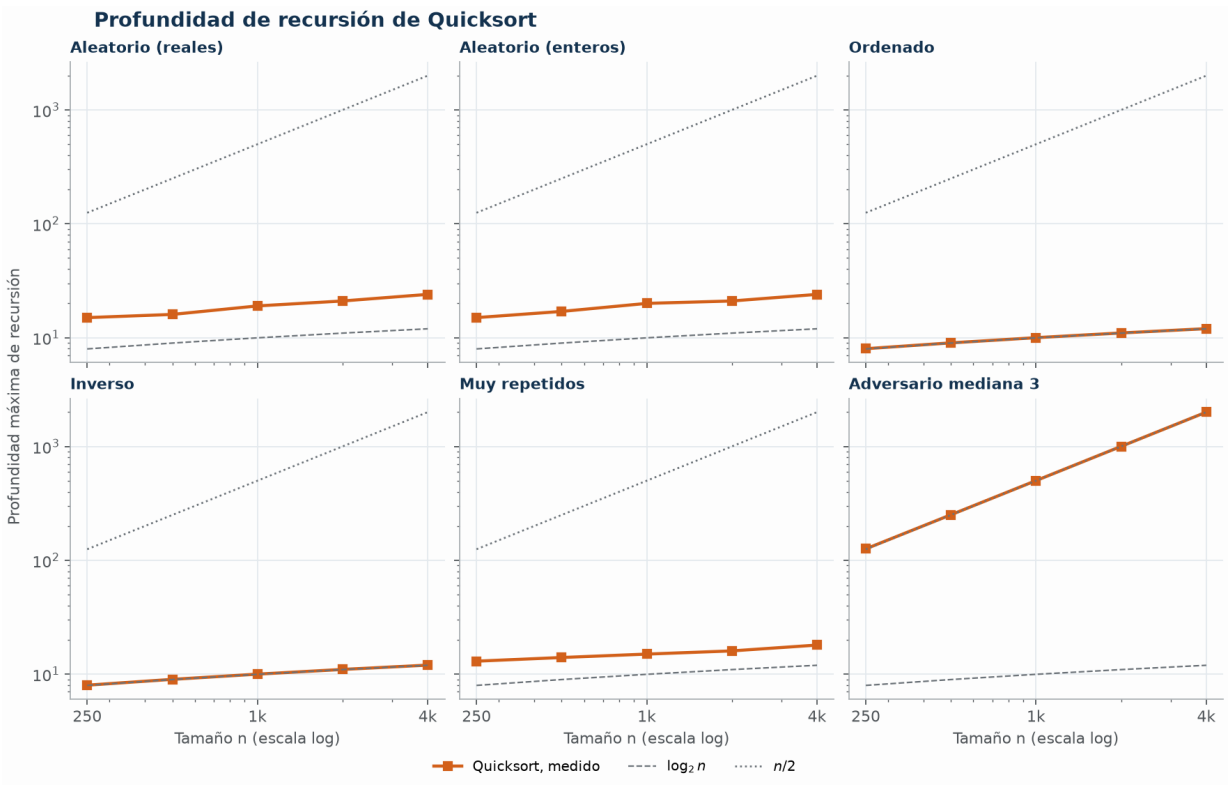



**Figura 7:** Profundidad máxima de recursión de Quicksort. En ordenado e inverso coincide con $\log_2 n$; en aleatorio es cerca del doble; en el adversario coincide con $n/2$.

## 7. Teoría contra medición

**Bubble Sort, conteos.** Coinciden exactamente con el examen. Con $n = 4000$: 3 999 comparaciones en ordenado, que es $n - 1$, y 7 998 000 comparaciones e intercambios en inverso, que es $n(n-1)/2$. En aleatorio hay 7 995 984 comparaciones, un poco menos que 7 998 000 por la salida temprana, y 4 033 288 intercambios, cerca de $n(n-1)/4 = 3\,999\,000$, que es el número esperado de inversiones de una permutación aleatoria.

**Bubble Sort, tiempos.** Con todos los tamaños, la pendiente en los casos cuadráticos va de 2.14 a 2.17, por encima del 2 teórico. El exceso viene de los tamaños pequeños. Con $n \geq 2000$ la pendiente baja a un rango de 2.03 a 2.06. Los conteos sí crecen como $n^2$, así que lo que cambia es el costo de cada operación: en aleatorio, el tiempo por comparación sube de unos 25 ns con $n = 250$ a unos 44 ns con $n = 4000$. La causa probable es la memoria caché, porque un arreglo mayor deja de caber en los niveles rápidos. Esto no se midió, así que queda como hipótesis.

**Bubble Sort, ordenado.** La pendiente es 1.05 con todos los tamaños y 1.01 con $n \geq 2000$, frente a 1 en teoría. Aquí los tiempos pequeños son de pocos microsegundos y pesa más el ruido.

**Quicksort, conteos.** La recurrencia del mejor caso del examen predice unas $n \log_2 n = 47\,863$ comparaciones con $n = 4000$. En ordenado se contaron 48 105, un factor de 1.01. En aleatorio se contaron 67 058, un factor de 1.40, porque las particiones no son exactas: la profundidad es 24, frente a 12.0 de $\log_2 n$. El orden de crecimiento se conserva, pero la constante es mayor.

**Quicksort, tiempos.** La pendiente en los casos no adversarios va de 1.11 a 1.13, cerca de la pendiente de 1.13 que tiene $n \log_2 n$ en este rango. Una pendiente mayor que 1 es lo que se espera del factor logarítmico. Con el adversario la pendiente es 2.03 y las comparaciones son 4 019 986, cerca de $n^2/4 = 4\,000\,000$. Este valor es la mitad de las $n^2/2$ del peor caso del examen, lo que concuerda con una profundidad de $n/2$: cada nivel descarta unos dos elementos en lugar de uno.

**Lo que la notación asintótica no predice.** Dos configuraciones con la misma complejidad pueden diferir mucho:

- Bubble Sort inverso tarda 1.27 veces lo que tarda aleatorio, porque intercambia en cada comparación (7 998 000 intercambios frente a 4 033 288).
- Bubble Sort con la entrada adversaria tarda menos que con aleatorio (3.93 s frente a 5.85 s). Esa entrada solo perjudica a Quicksort. Bubble Sort hace menos comparaciones (5 997 000) y menos intercambios (2 662 731).
- Quicksort inverso tarda la mitad que aleatorio (5.17 ms frente a 10.4 ms), con particiones exactas y solo 2 000 intercambios frente a 11 568.
- Con muchos repetidos, Quicksort hace más intercambios (18 161) porque los punteros se detienen en los iguales, y a cambio la profundidad baja a 18.

**Profundidad en la animación.** El contador de la animación cuenta las llamadas que hacen trabajo. Por eso puede dar 1 menos que este experimento, que también cuenta las llamadas que regresan de inmediato.

## 8. Cuándo conviene cada algoritmo

**Quicksort** ganó en todas las configuraciones menos la ordenada. Su ventaja crece con $n$, salvo con el adversario, donde se estabiliza. Con $n = 16000$ fue 562 veces más rápido en aleatorio, 1435 veces en inverso y 639 veces con repetidos. Ya con $n = 250$, el menor tamaño medido, la ventaja en aleatorio es de 8.1 veces. Su límite es la entrada adversaria: tarda 106 veces más que

en aleatorio, aunque sigue siendo 3.5 veces más rápido que Bubble Sort. Además usa pila de recursión, de $\Theta(\log n)$ en el mejor caso hasta $\Theta(n)$ en el peor. Con la entrada adversaria hubo que subir el límite de recursión de Python.

**Bubble Sort** solo ganó con el arreglo ya ordenado: 0.403 ms frente a 5.01 ms con $n = 16000$, unas 12.4 veces más rápido, y sin escribir nada. Su memoria adicional es $O(1)$. Fuera de ese caso, la medición no lo favorece en ningún tamaño desde $n = 250$.

## 9. Limitaciones

- Todo se midió en un solo equipo, con un intérprete de Python. Los tiempos absolutos no se trasladan a otra máquina; las pendientes y razones son más estables.
- No se midieron arreglos casi ordenados ni tamaños menores que 250, así que este informe no afirma nada medido sobre ellos.
- La memoria no se midió directamente. La profundidad de recursión es la única medida relacionada.
- Las configuraciones ordenado, inverso y adversario son deterministas: sus repeticiones miden solo el ruido del equipo, no distintas entradas.

## Referencia

McIlroy, M. D. (1999). A killer adversary for quicksort. *Software: Practice and Experience*, 29(4), 341-344.